

Whitelist Manager

for Splunk Enterprise Security

Project Documentation

Architecture, Technologies & File Reference

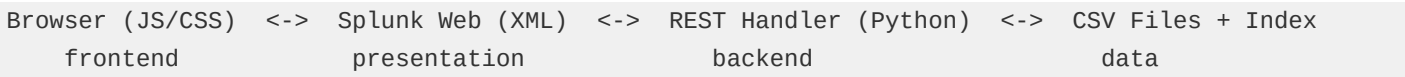
Version 1.0.0

February 2026

1. Architecture Overview

The Whitelist Manager is a Splunk application for managing CSV whitelist files used by 300+ detection rules in Splunk Enterprise Security. It provides a web-based editable table with Git-style audit trail, RBAC enforcement, and dual audit storage.

The application has four main layers:



Each layer uses the technology best suited for its purpose. Splunk enforces specific technology choices in many areas (e.g., Python for REST handlers, Simple XML for dashboards, INI-style .conf files for configuration). Where Splunk does not prescribe a choice, we use standard web technologies (JavaScript, CSS, JSON).

2. Splunk Configuration Files (.conf)

Splunk uses INI-style configuration files to control every aspect of the platform. These files are placed in the app's default/ directory and are read by Splunk on startup. There is no alternative to .conf files - this is Splunk's native configuration language.

default/app.conf

App identity and metadata. Defines the app name (wl_manager), version (1.0.0), author, description, and package ID. Splunk uses this to display the app in the Apps menu and to identify it during installation/upgrades.

default/restmap.conf

Registers the custom REST API endpoint. This file tells Splunk: 'When someone calls /custom/wl_manager, route the request to the Python class WhitelistHandler in wl_handler.py.'

Key settings:

- match = /custom/wl_manager - the URL path for the endpoint
- script = wl_handler.py - the Python file containing the handler
- handler = wl_handler.WhitelistHandler - the Python class to instantiate
- scripttype = persist - keeps the Python process alive (faster than spawning a new process per request)
- requireAuthentication = true - only authenticated Splunk users can call this endpoint
- python.version = python3 - use Python 3 (Splunk also supports Python 2 for legacy apps)

default/web.conf

Exposes the REST endpoint through Splunk Web (port 8000). Without this file, the endpoint would only be accessible via the management port (8089). The web.conf 'expose' stanza acts as a reverse proxy, allowing the browser-based JavaScript to call the REST API.

default/indexes.conf

Creates the wl_audit index for storing audit trail events. Configures storage paths, a 3-year retention policy (frozenTimePeriodInSecs), and a 1GB data cap (maxTotalDataSizeMB). Having a dedicated index allows separate access controls and retention from other data.

default/authorize.conf

Defines two custom RBAC (Role-Based Access Control) roles:

- wl_editor - can read and write whitelists and view the audit trail
- wl_viewer - read-only access to whitelists and audit trail

Both roles inherit from Splunk's built-in 'user' role and are granted access to the wl_audit index. The REST handler checks these roles before allowing write operations.

default/transforms.conf

Registers rule_csv_map.csv as a Splunk lookup table. This allows dashboard searches to query it using the SPL

command '| inputlookup rule_csv_map'. Without this registration, the CSV would just be a file on disk with no Splunk integration.

3. XML Dashboards (Simple XML)

Splunk uses Simple XML as its dashboard definition language. It is a declarative format specific to Splunk - you describe what you want (dropdowns, tables, charts), and Splunk renders them with built-in handling for authentication, search execution, and interactivity.

default/data/ui/nav/default.xml

The navigation bar (app menu). Defines two tabs: 'Whitelist Manager' (the main editing dashboard) and 'Audit Trail' (the change history dashboard). Also includes a link to Splunk's built-in search page for ad-hoc queries.

default/data/ui/views/whitelist_manager.xml

The main interactive dashboard. It uses several Splunk XML features:

- Two `<input type="dropdown">` elements with the 'search' attribute - these run SPL queries to populate the dropdown options from the rule_csv_map lookup
- Token system (`$rule_token$`, `$csv_token$`) - when you select a detection rule, it sets a token that the second dropdown uses to filter its options. This is how the dropdowns are 'codependent'
- A `<search>` element with a 'depends' attribute that fires when csv_token is set, resolving the app_context for the selected CSV
- An `<html>` panel that serves as a container where JavaScript (whitelist_manager.js) renders the editable table. Simple XML cannot create editable tables natively

SPL (Search Processing Language) is Splunk's query language, similar to SQL but designed for time-series event data. Example: `| inputlookup rule_csv_map | dedup rule_name | sort rule_name` reads the CSV, removes duplicate rule names, and sorts them alphabetically.

default/data/ui/views/audit.xml

The audit trail dashboard. More complex than the main dashboard, it includes:

- Time range picker and dropdown filters for analyst/rule selection
- Single-value panels showing total changes, rows added, and rows removed (using stats SPL command)
- A changelog table with drilldown - clicking a row shows the detailed diff
- Unified diff viewer panel that displays Git-style diffs stored in the audit events

The 'spath' SPL command is used heavily here - it extracts fields from JSON-formatted audit events that were indexed as raw strings.

4. Python Backend

Python is the only language Splunk supports for custom REST endpoints. The handler runs as a long-lived process inside Splunk's Python environment (PersistentServerConnectionApplication). Splunk calls the `handle()` method for each incoming HTTP request.

`bin/wl_handler.py` - REST Handler (core of the app)

This is the server-side engine. It receives HTTP requests from the JavaScript frontend, reads/writes CSV files, computes diffs, enforces RBAC, and writes audit events.

Python standard libraries used and why:

Library	Purpose
json	Parse incoming REST requests and format JSON responses (REST APIs use JSON)
csv	Read/write CSV files safely - handles quoting, escaping, encoding correctly
difflib	Generate unified diffs (the '--- before / +++ after' Git-style format)
logging	Rotating file logger for audit trail backup (10MB files, 5 backups)
os / sys	File paths, environment variables (SPLUNK_HOME), stderr output
datetime	UTC timestamps (ISO-8601) for audit events
urllib / ssl	HTTPS POST to Splunk's receivers/simple endpoint to index audit events

Splunk-internal libraries (built into Splunk's Python, not installable via pip):

Library	Purpose
splunk.persistconn	Base class for persistent REST handlers (PersistentServerConnectionApplication)
splunk.rest	Query Splunk's REST API internally (used to fetch user roles for RBAC)

Key functions in the handler:

- `_safe_filename()` - prevents path traversal attacks (e.g., `'../../etc/passwd'` as filename)
- `_resolve_csv_path()` - locates CSV files across different Splunk apps via `app_context`
- `_compute_diff()` - compares old vs new CSV, returns added/removed rows + unified text diff
- `_get_roles()` - fetches user roles from Splunk's authentication API (roles are NOT in the session)
- `_index_audit()` - writes audit events to Splunk index using Python's built-in `urllib` (no SDK needed)
- `handle()` - entry point called by Splunk for every request; routes to GET or POST handlers

Design decision: we use Python's built-in `urllib.request` instead of the external Splunk SDK (`splunklib`) for indexing audit events. This means the app works without any extra package installation - a key requirement since Splunk admins may not allow pip installs on production servers.

`bin/wl_wrapper.py` - CLI Wrapper

A standalone command-line tool requested by the Splunk Infrastructure Team for bulk operations and scripting. It provides the same functionality as the REST handler but via terminal commands.

Commands: `list`, `add`, `remove`, `diff`

Additional Python libraries used:

- `argparse` - parses CLI commands and arguments with built-in help text and validation

- ANSI escape codes - colorized terminal output for diffs (green=added, red=removed)

The CLI wrapper writes to the same audit log file as the REST handler, so all changes (whether made via the web UI or terminal) appear in a single audit trail.

5. Frontend (JavaScript + CSS)

appserver/static/whitelist_manager.js

JavaScript is needed because Splunk's XML dashboards cannot create editable tables. This file runs in the browser and handles all user interaction: loading data, rendering the editable table, saving changes, and displaying diffs.

Technologies and frameworks used:

Technology	Purpose
require.js (AMD)	Splunk's module loader - loads Splunk MVC components (bundled with Splunk)
jQuery (\$)	DOM manipulation - builds HTML tables, handles clicks, AJAX calls (bundled)
Underscore.js (_)	Utility functions for arrays/objects (bundled with Splunk)
Splunk MVC	Token model listeners - watches dropdown changes and triggers data loading
\$.ajax()	Makes REST API calls to our Python handler (GET to load, POST to save)
Splunk.util	make_url() generates correct URLs with Splunk's locale prefix

Key features built in JavaScript:

- Renders an HTML <table> with contenteditable cells for inline editing
- Add Row / Remove Row buttons for managing whitelist entries
- Mandatory comment field before save (required for audit trail)
- After save, displays the diff returned by the Python handler
- Token listeners that respond to dropdown selections and load the corresponding CSV data

appserver/static/whitelist_manager.css

CSS styles for the editable table, buttons, alerts, and diff panel. All class names are prefixed with 'wl-' to avoid collisions with Splunk's own CSS framework.

Key style sections:

- Dark header row and hover effects for the editable table
- Green/red highlighting for diff display (mimics GitHub's diff view)
- Monospace font for unified diff text (like a terminal)
- Colored alert styles: red (error), green (success), blue (info), orange (warning)

6. Data Files

lookups/rule_csv_map.csv

The master mapping table - the 'source of truth' that connects detection rules to their CSV whitelist files. When you select a rule in the dropdown, the app looks up which CSVs belong to it from this file.

Three columns:

- rule_name - the detection rule identifier (e.g., DR45_suspicious_login)
- csv_file - the CSV filename (e.g., DR45_whitelist_users.csv)
- app_context - which Splunk app contains the CSV (e.g., SplunkEnterpriseSecuritySuite)

Why CSV? Splunk's lookup system is built around CSV files stored in \$SPLUNK_HOME/etc/apps/<app>/lookups/. They can be queried with '| inputlookup' in SPL.

metadata/default.meta

Splunk's permission file. Controls which roles can read and write which app objects (views, lookups, REST endpoints). This is RBAC at the Splunk platform layer, separate from the application-level RBAC in the Python handler.

7. Docker Testing Environment

Docker allows us to run a full Splunk instance locally for testing without affecting production. The containerized Splunk is identical to production, ensuring our tests are realistic.

docker-compose.yml

Declares the containerized Splunk environment using Docker Compose (YAML format).

- Image: splunk/splunk:9.3.1 (official Splunk Enterprise image)
- Ports: 8000 (Splunk Web), 8089 (management API), 8088 (HTTP Event Collector)
- Bind mounts: maps local app directories into the container for live code reloading
- Named volume (splunk_var): persists Splunk data across container restarts
- Environment: pre-configures admin password, accepts license, enables HEC

.docker/default.yml

Ansible-based Splunk configuration. The official Splunk Docker image uses Ansible internally to configure itself on first boot. This YAML file pre-creates the wl_audit index so it exists before any tests run.

8. Build & Test Scripts (Bash)

Bash is the standard language for build and CI/CD automation. These scripts work on Linux (GitHub Actions CI) and Windows (Git Bash).

scripts/validate.sh - Pre-flight Validation

Runs 8 categories of checks before packaging:

- 1. Required files - verifies all expected files exist
- 2. app.conf structure - checks version, package ID, launcher stanza
- 3. Python syntax - calls 'python -c compile(...)' to check for syntax errors
- 4. XML well-formedness - parses dashboard XMLs with Python's ElementTree
- 5. Security - greps for hardcoded passwords, tokens, dangerous functions (eval, exec, os.system)
- 6. CSV validation - checks that lookup CSVs have proper headers
- 7. Dangerous patterns - detects risky code patterns
- 8. Forbidden files - AppInspect rules: no .pyc, __pycache__, or .git

scripts/package.sh - SPL Packaging

Builds the .spl file for distribution. An .spl file is simply a .tar.gz archive containing the app directory as the root entry. Splunk extracts it into \$SPLUNK_HOME/etc/apps/ during installation.

Steps: clean artifacts -> run validation -> create tar.gz -> generate SHA-256 checksum

Output: dist/wl_manager-<version>.spl + .sha256 integrity file

scripts/test_integration.sh - Integration Tests

11 tests that run against the containerized Splunk instance using curl (HTTP client). Tests the app the same way a real user would - by sending HTTP requests.

Test categories:

- Connectivity - Splunk management API and web interface are reachable
- App installation - wl_manager app is installed and wl_audit index exists
- REST GET - get_rules and get_mapping return valid JSON with expected data
- REST POST - save_csv writes CSV, returns diff, and audit event appears in index
- Dashboards - whitelist_manager and audit views are accessible

9. CI/CD Pipelines (GitHub Actions)

GitHub Actions provides automated build and test pipelines triggered by Git events. Workflows are defined in YAML files under `.github/workflows/`.

`.github/workflows/ci.yml` - Continuous Integration

Triggers: every push to main branch and every pull request.

Steps:

- 1. Checkout code (actions/checkout@v4)
- 2. Set up Python 3.9 (actions/setup-python@v5)
- 3. Run `validate.sh` - all 28 checks must pass
- 4. Run `package.sh` - builds the `.spl` file
- 5. Upload `.spl` as artifact (downloadable for 30 days)

If any step fails, the pipeline stops and the commit/PR is marked as failed. This prevents broken code from being merged into the main branch.

`.github/workflows/release.yml` - Automated Releases

Triggers: when you publish a GitHub release (create a new tag like `v1.1.0`).

Steps:

- 1. Validate and build `.spl` package
- 2. Attach the `.spl` and `.sha256` files to the GitHub release

This means your Splunk admin can download the latest `.spl` directly from the GitHub Releases page - no manual packaging needed.

Other Project Files

Makefile

A convenience wrapper providing short commands (`make test`, `make package`, `make docker-up`) that call the underlying bash scripts. Useful on Linux/macOS; on Windows, the bash scripts can be called directly.

`.gitignore`

Tells Git which files to exclude from version control: build output (`dist/`, `.spl`), Python bytecode (`.pyc`, `__pycache__`), OS files (`.DS_Store`, `Thumbs.db`), Splunk runtime files (`local/`), and IDE files (`.vscode/`, `.idea/`).

`.dockerignore`

Tells Docker which files to exclude when building images: development files (scripts, tests, `.git`, `.github`, markdown docs). Keeps the Docker context small and fast.

`tests/sample_whitelist.csv`

Sample CSV file with realistic test data (hostnames, usernames, commands). Used by integration tests to verify CSV read/write operations.

10. Technology Summary

Technology	Where Used	Why
Python 3	REST handler, CLI	Splunk's only language for custom REST endpoints
Simple XML	Dashboards	Splunk's native dashboard framework (handles auth, search)
JavaScript	Frontend controller	Needed for editable tables (XML can't do this)
CSS	Styling	Visual design for table, buttons, diffs, alerts
SPL	Dashboard queries	Splunk's query language (only way to query Splunk data)
Bash	Build/test scripts	Standard for CI/CD; works on Linux and Git Bash
Docker + YAML	Local testing	Isolated Splunk instance for safe testing
GitHub Actions	CI/CD pipelines	Automated validation on push, packaging on release
CSV	Data storage	Splunk's native lookup format used by 300+ rules
JSON	REST API + audit	Industry standard for API communication
INI (.conf)	Splunk config	Splunk's native config format (no alternative)
Git	Version control	Track changes, collaborate, distribute via releases